Ad Soyad: _____	Tarih: _____
Proje Ödevi – RESYS – *Restoran Sipariş ve Stok Yönetimi*	

Proje Hakkında

Bu projede bir **Restoran Sipariş ve Stok Yönetim Sistemi (RESYS)** geliştireceksiniz. Sistem; menü yönetimi, sipariş takibi, mutfak ekranı ve stok kontrolü işlevlerini içerecektir. Proje dönem boyunca 4 fazda geliştirilecek ve her faz bir öncekinin üzerine inşa edilecektir.

- **Faz 1 (Hafta 1–3):** STL kapsayıcıları ve modern C++ ile veri katmanı
- **Faz 2 (Hafta 4–8):** Qt6 ile grafiksel kullanıcı arayüzü
- **Faz 3 (Hafta 9–11):** Çok kanallı programlama ile performans iyileştirmesi
- **Faz 4 (Hafta 12–14):** Ağ programlama ile istemci/sunucu mimarisi

Süre: 14 hafta (dönem boyunca) **Başlangıç kodu:** `out/Projects/RESYS/`

Teknoloji Gereksinimleri

Bu proje **C++20** standardı ve **Qt6** çerçevesi kullanılarak geliştirilmelidir. Aşağıdaki araçların sisteminizde kurulu olması gerekir:

- **Derleyici:** g++ veya clang++ (C++20 desteği ile)
- **Qt6:** Qt 6.5+ (Widgets, PrintSupport, Charts modülleri)
- **CMake:** 3.20 veya üzeri

Faz 1 derleme:

```
g++ -std=c++20 -o resys src/main.cpp
```

Faz 2+ derleme (Qt ile):

```
mkdir build && cd build
cmake .. -DCMAKE_PREFIX_PATH=/path/to/Qt/6.x
cmake --build .
```

1 Alan Modeli

Sisteminiz aşağıdaki temel varlıkları içerecektir:

Varlık	Anahtar	Alanlar
`MenuItem`	`urun_id`	ürün adı, kategori, fiyat
`SiparisKalemi`	–	ürün id, adet, notlar
`Siparis`	`siparis_id`	masa no, tarih/saat, kalemler (`vector<SiparisKalemi>`), durum
`StokMalzeme`	`malzeme_id`	malzeme adı, miktar, birim, minimum stok

Durum Enum'ları

```
enum class SiparisDurum {
    Hazirlaniyor,    // Siparis mutfakta hazirlaniyor
    Hazir,           // Siparis hazir, servise bekliyor
    TeslimEdildi,    // Siparis masaya teslim edildi
    Iptal            // Siparis iptal edildi
};
```

Örnek Struct Tanımı

```
struct MenuItem {
    int urun_id{};
    std::string urun_adi;
    std::string kategori;
    double fiyat{};
};
```

Varlık İlişkileri

- Bir `Siparis` birden fazla `SiparisKalemi` içerebilir (1:N)
- Her `SiparisKalemi`, bir `MenuItem`'a referans verir (N:1)
- `StokMalzeme` bağımsız olarak takip edilir; menü ile dolaylı ilişkilidir

2 Faz 1: STL ve Veri Katmanı (Hafta 1–3)

Faz 1 Hedefi

Bu fazda projenin veri katmanını oluşturacaksınız. Sipariş kuyruğu için `std::queue`, stok takibi için `std::map`, menü sıralama ve filtreleme için STL algoritmaları kullanılacaktır. **Bu fazda henüz Qt kullanılmayacaktır;** yalnızca C++20 standart kütüphanesi kullanılacaktır.

Görev 1.1 – Varlık Struct'ları

`include/varliklar.h` dosyasında aşağıdaki struct'ları tanımlayın:

- `MenuItem`, `SiparisKalemi`, `Siparis`, `StokMalzeme` struct'ları
- `SiparisDurum` enum class'ı
- Her struct için `operator«` (akış çıkış operatörü)
- En az bir alanda `std::optional` kullanımı (örneğin sipariş notları)

Görev 1.2 – Depo Şablon Sınıfı

`include/depo.h` dosyasındaki `Depo<K,V>` şablon sınıfını tamamlayın:

- `ekle(anahtar, deger)`: Yeni kayıt ekler
- `bul(anahtar) -> std::optional<V>`: Anahtar ile arama yapar
- `sil(anahtar) -> bool`: Kaydı siler
- `tumunu_al() -> const std::map<K,V>&`: Tüm kayıtları döner
- `filtrele(kosul) -> std::vector<V>`: Lambda ile filtreleme

Görev 1.3 – STL Algoritmaları

Aşağıdaki sorguları STL algoritmaları ve/veya C++20 ranges kullanarak yazın:

- Belirli bir kategorideki menü öğelerini listeleme
- Sipariş kuyruğundaki bekleyen siparişleri sırayla işleme (`std::queue`)
- Menü öğelerini fiyata göre sıralama
- Bir masanın toplam hesabını hesaplama
- Stok seviyesi minimum altına düşen malzemeleri bulma

Görev 1.4 – Dosya İşlemleri

Verileri ikili dosyada saklayın ve okuyun:

- `dosyaya_kaydet(dosya_yolu, depo)`: Depo içeriğini dosyaya yazar
- `dosyadan_oku(dosya_yolu) -> Depo`: Dosyadan veri okur
- Program kapanırken otomatik kaydetme, açılırken otomatik yükleme

İpucu: Sipariş Kuyruğu

`std::queue` ile sipariş işleme örneği:

```
std::queue<Siparis> siparis_kuyrugu;
siparis_kuyrugu.push(yeni_siparis);

while (!siparis_kuyrugu.empty()) {
    auto& siradaki = siparis_kuyrugu.front();
    // Siparisi hazirla...
    siparis_kuyrugu.pop();
}
```

3 Faz 2: Qt6 ile Grafiksel Arayüz (Hafta 4–8)

Faz 2 Hedefi

Bu fazda Faz 1'de oluşturduğunuz veri katmanının üzerine Qt6 ile bir masaüstü arayüzü inşa edeceksiniz. **Qt6 Widgets** modülü kullanılacak ve **CMake** ile derlenecektir.

Görev 2.1 – Ana Pencere ve Masa Düzeni

`QMainWindow` tabanlı ana pencere oluşturun:

- Menü çubuğu: Dosya (Kaydet, Yükle, Çıkış), Sipariş (Yeni Sipariş), Stok, Yardım
- Araç çubuğu: Sık kullanılan işlemler için butonlar
- Masa düzeni görsel haritası: Restorandaki masaları temsil eden buton ızgarası
- Durum çubuğu: Aktif sipariş sayısı ve son işlem bilgisi

Görev 2.2 – Sipariş Giriş Ekranı

Sipariş alma için `QDialog` tabanlı giriş formu oluşturun:

- Masa seçimi (`QSpinBox` veya masa butonundan otomatik)
- Menü listesinden ürün seçimi (`QListWidget` veya `QTableView`)
- Adet girişi ve sipariş notu alanı
- Sipariş özeti ve toplam tutar gösterimi
- Form doğrulama: Boş sipariş kontrolü, geçerli adet kontrolü

Görev 2.3 – Mutfak Ekranı

Aktif siparişleri gösteren mutfak ekranı oluşturun:

- `QAbstractTableModel` alt sınıfı ile sipariş listesi
- Sipariş durumuna göre renklendirme (hazırlanıyor: sarı, hazır: yeşil)
- "Hazır" butonuyla sipariş durumunu güncelleme
- `QSortFilterProxyModel` ile duruma göre filtreleme

Görev 2.4 – Raporlama ve Grafik

- Pencere boyutu ve konumunu `QSettings` ile kaydetme/yükleme
- JSON formatında menü ve stok dışa aktarımı (`QJsonDocument`)
- Günlük ve aylık ciro grafiği (`QChart`): Tarih bazlı gelir dağılımı

İpucu: Sinyal/Slot Bağlantısı

Qt6'da yeni bağlantı söz dizimini kullanın (derleme zamanı kontrolü):

```
connect(masaButon, &QPushButton::clicked,
        this, &MainWindow::siparisAl);
```

4 Faz 3: Çok Kanallı Programlama (Hafta 9–11)

Faz 3 Hedefi

Bu fazda uygulamanızı çok kanallı hale getirerek mutfak ekranının gerçek zamanlı güncellenmesini ve arka planda stok uyarılarının verilmesini sağlayacaksınız. `std::thread`, `std::mutex` ve Qt'nin `QThread` mekanizması kullanılacaktır.

Görev 3.1 – Gerçek Zamanlı Mutfak Ekranı

Mutfak ekranını gerçek zamanlı güncelleyin:

- Yeni sipariş geldiğinde mutfak ekranının otomatik güncellenmesi
- Sipariş durumu değişikliklerinin anlık yansıtılması
- Arka planda stok seviyesi kontrolü ve minimum altı uyarıları

Görev 3.2 – Senkronizasyon

- Paylaşılan sipariş kuyruğunu `std::mutex` / `std::lock_guard` ile koruma
- İlerleme bildirimi: Stok kontrol izleğinden GUI'ye sinyal gönderme
- İptal mekanizması: Kullanıcı stok kontrolünü durdurabilmeli

İpucu: QThread ile Stok Kontrolü

```
class StokKontrolcu : public QObject {
    Q_OBJECT
signals:
    void stokUyarisi(const QString& malzeme, double kalan);
    void kontrolTamamlandi();
    void hataOlustu(const QString& mesaj);
};
```

5 Faz 4: Ağ Programlama (Hafta 12–14)

Faz 4 Hedefi

Bu fazda uygulamanızı istemci/sunucu mimarisine dönüştüreceksiniz. Merkezi kasa sunucusu tüm verileri yönetecek, garson tabletleri birer istemci olarak sipariş gönderecek ve mutfak ekranı ayrı bir istemci olarak siparişleri gösterecektir. **Qt Network** modülü ve **TCP soketleri** kullanılacaktır.

Görev 4.1 – TCP Sunucu (Merkezi Kasa)

`QTcpServer` ile merkezi kasa sunucusu oluşturun:

- Birden fazla garson tabletini ve mutfak ekranını eş zamanlı kabul etme
- JSON tabanlı mesaj protokolü (istek/yanıt formatı)
- Sipariş çakışma kontrolü (aynı masaya eş zamanlı sipariş)

Görev 4.2 – TCP İstemciler

`QTcpSocket` ile istemci uygulamaları oluşturun:

- **Garson tableti:** Sunucuya bağlanma, sipariş gönderme, hesap isteme
- **Mutfak ekranı:** Sunucudan sipariş alma, durum güncelleme
- Bağlantı kopması durumunda yeniden bağlanma denemesi

İpucu: JSON Mesaj Protokolü

```
// Istek formati (garson -> sunucu)
{
    "tip": "yeni_siparis",
    "masa_no": 5,
    "kalemler": [
        {"urun_id": 101, "adet": 2, "not": "az pismis"}
    ]
}

// Yanit formati (sunucu -> garson)
{
    "durum": "basarili",
    "mesaj": "Siparis alindi",
    "siparis_id": 42
}
```

6 Değerlendirme Kriterleri

Faz	Puan	Değerlendirme Alanları
Faz 1: STL ve Veri Katmanı	25	Struct tasarımı, Depo şablonu, STL algoritmaları, dosya I/O, `optional`/`variant` kullanımı
Faz 2: Qt GUI	35	Masa düzeni, sipariş formu, mutfak ekranı, sinyal/slot, QSettings, QChart, kod kalitesi
Faz 3: Çok Kanallı	20	Thread kullanımı, mutex/lock_guard, gerçek zamanlı güncelleme, stok uyarı mekanizması
Faz 4: Ağ Programlama	20	TCP sunucu/istemci, JSON protokol, çoklu istemci desteği, hata yönetimi
Toplam	**100**	

7 Teslim Edilecekler

Her faz sonunda aşağıdaki dosyaları teslim edin:

Faz 1 Teslimi (Hafta 3 sonu)

- `include/varliklar.h` – Varlık struct'ları ve enum'lar
- `include/depo.h` – Depo şablon sınıfı
- `src/main.cpp` – Test ve demonstrasyon kodu
- `CMakeLists.txt` – Derleme yapılandırması

Faz 2 Teslimi (Hafta 8 sonu)

- Faz 1 dosyaları + Qt GUI kaynak dosyaları
- `src/main_window.h/cpp` – Ana pencere ve masa düzeni
- `src/siparis_model.h/cpp` – Sipariş tablo modeli
- Güncellenmiş `CMakeLists.txt` (Qt6 bağımlılıkları ile)

Faz 3 Teslimi (Hafta 11 sonu)

- Faz 2 dosyaları + thread sınıfları
- En az bir arka plan görevi uygulaması (stok kontrolü veya mutfak güncelleme)

Faz 4 Teslimi (Hafta 14 – Final)

- Tüm kaynak kodlar
- Sunucu (merkezi kasa) ve istemci (garson tableti, mutfak ekranı) uygulamaları
- Kısa kullanım kılavuzu (README.md)
- Proje sunumu (5–10 dakika)